# Mini-Project Report

Team members: Avleen Kaur Virdi, Luu Dang Thanh Ha

## I/ Step 1 – Determine Requirements:

1. 200 – OK
   Code 200 is sent when the server receives a valid request and successfully retrieves or generates the requested resource.
   Conditions include:
   Request line has a valid structure: METHOD PATH VERSION.
   Methods are supported (GET or HEAD).
   HTTP version is supported (HTTP/1.0 or HTTP/1.1).
   Requested file exists in ./www, or the route is a valid dynamic one (/add, /cart.html).
   The path is not restricted (e.g., not /admin or /config).

   Example request:
   GET /index.html HTTP/1.1
   GET /add?item=apple HTTP/1.1
   GET /cart.html HTTP/1.1
   HEAD /index.html/1.1

2. 304 - Not Modified
   Code 304 is sent when the client sends a conditional GET request with an If-Modified-Since header, and the file modification time is not newer than that timestamp. The server therefore returns headers only to save bandwidth.
   Example request:
   GET /index.html HTTP1/1
   Host: localhost:8080
   If-Modified-Since: Fri, 24 Oct 2025 12:00:00 GMT

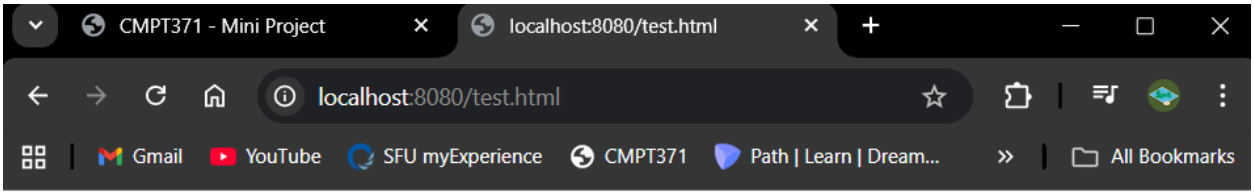
3. 403 – Forbidden
   Code 403 is sent when the client attempts to access a restricted directory or uses an unsupported method.
   Restricted paths: /admin/..., /config/... (blocked for security).
   Unsupported methods: any method other than GET or HEAD (e.g., POST)
   Example request:
   GET /admin/manager.html HTTP/1.1
   POST /index.html HTTP/1.1

4. 404 – Not Found
   Code 404 is sent when the requested resource cannot be located or is not a recognized route.
   Cases include:
      Static file missing: the requested file does not exist in ./www.
      Invalid dynamic route: the request path is not one of the supported dynamic endpoints (/add, /cart.html)
   Example request:
   GET /missing.html HTTP/1.1
   GET /randompage HTTP/1.1

5. 505 – HTTP Version Not Support
   Code 505 is sent when the client uses an unsupported HTTP version in the request line. The server only supports HTTP/1.0 and HTTP/1.1.
   Example request:
   GET /test.html HTTP/2.0

## II/ Step 2 – Build Minimal Web Server & Test:

a. Implementation (in server.py)
b. Test whether server is working in browser




Congratulations! Your Web Server is Working!

c. Testing request:
   1. 200 - OK
      Case 1: Valid file in ./www
      Purpose: Verify normal successful response for existing files.
      Test command:
         curl -v http://localhost:8080/add?item=apple
      Expected response:
      HTTP/1.1 200 OK
      Content-Type: text/html
      Conten-Length: ...
      <html> ... </html>

Explanation:

The file “index.html” exists in ./www

The method (GET) and HTTP version (HTTP/1.1) are valid.

The server therefore returns a full 200 OK response with file content.

```
D:\thanh\Fall2025\CMPT371\MiniServer>curl -v http://localhost:8080/index.html
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
*   Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080
* using HTTP/1.x
> GET /index.html HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.14.1
> Accept: */*
>
< HTTP/1.1 200 OK
< Date: Sat, 25 Oct 2025 21:03:02 GMT
< Server: MiniCartServer/1.0
< Content-Type: text/html
< Transfer-Encoding: chunked
< Connection: close
<
```

Case 2: Dynamic route successfully generated cart page

Purpose: Verify that the server correctly handles a recognized dynamic route (/add) and generates an HTML cart page dynamically.

Test command:

curl -v http://localhost:8080/add?item=apple

Expected response:

HTTP/1.1 200 OK

Content-Type: text/html

Conten-Length: ...

<html> ... </html>

Explanation:

The file “index.html” exists in ./www

The method (GET) and HTTP version (HTTP/1.1) are valid.

The server therefore returns a full 200 OK response with file content.

```
D:\thanh\Fall2025\CMPT371\MiniServer>curl -v http://localhost:8080/add?item=apple
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
*   Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080
* using HTTP/1.x
> GET /add?item=apple HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.14.1
> Accept: */*
>
< HTTP/1.1 200 OK
< Date: Sat, 25 Oct 2025 21:11:18 GMT
< Server: MiniCartServer/1.0
< Content-Type: text/html
< Content-Length: 117
< Connection: close
<
<html><body><h1>Shopping Cart</h1><ul><li>apple: 5</li></ul><a href='/index.html'>Continue shopping
o close, stop send
* abort upload
* shutting down connection #0
```

2. 304 - Not Modified

   Purpose: Test conditional GET using an “If-Modified-Since" header.

   Test command:

   curl -v -H "If-Modified-Since: Fri, 25 Oct 2025 12:35:00 GMT"
   http://localhost:8080/test.html

   Expected response:

   HTTP/1.1 304 Not Modified

   Date: Fri, 25 Oct 2025 ...

   Server: MiniCartServer/1.0

   Connection: close

   Explanation: If the file’s last modification time is ≤ the time in the header, the server skips sending the body and returns 304 Not Modified.

```
D:\thanh\Fall2025\CMPT371\MiniServer>curl -v -H "If-Modified-Since: Fri, 25 Oct 2025 12:35:00 GMT" http://localhost:8080/test.html
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
*   Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080
* using HTTP/1.x
> GET /test.html HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.14.1
> Accept: */*
> If-Modified-Since: Fri, 25 Oct 2025 12:35:00 GMT
>
* Request completely sent off
< HTTP/1.1 304 Not Modified
< Date: Sat, 25 Oct 2025 21:02:20 GMT
< Server: MiniCartServer/1.0
< Content-Type: text/html
< Content-Length: 0
< Connection: close
<
* shutting down connection #0
```

3. 403 – Forbidden

   Case 1: Path blocked by server

   Purpose: Check accessing restricted folders is blocked.

   Test command:

curl -v http://localhost:8080/admin/secret.html

Expected response:

HTTP/1.1 403 Forbidden

Explanation: The path /admin/... matches a restricted prefix in the code.

```
D:\thanh\Fall2025\CMPT371\MiniServer>curl -v http://localhost:8080/admin/manager.html
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
*   Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080
* using HTTP/1.x
> GET /admin/manager.html HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.14.1
> Accept: */*
>
< HTTP/1.1 403 Forbidden
< Date: Sat, 25 Oct 2025 21:06:06 GMT
< Server: MiniCartServer/1.0
< Content-Type: text/html
< Content-Length: 40
< Connection: close
<
<h1>403 Forbidden - Access denied</h1>* we are done reading and this is set to close, stop send
* abort upload
* shutting down connection #0
```

Case 2: Unsupported HTTP method

Purpose: Check using unsupported methods is blocked.

Test command:

curl -v -X POST http://localhost:8080/index.html

Expected response:

HTTP/1.1 403 Forbidden

Explanation: The POST method is not allowed (GET and HEAD only).

```
D:\thanh\Fall2025\CMPT371\MiniServer>curl -v -X POST http://localhost:8080/index.html
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
*   Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080
* using HTTP/1.x
> POST /index.html HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.14.1
> Accept: */*
>
< HTTP/1.1 403 Forbidden
< Date: Sat, 25 Oct 2025 21:05:02 GMT
< Server: MiniCartServer/1.0
< Content-Type: text/html
< Content-Length: 45
< Connection: close
<
<h1>403 Forbidden - Unsupported Method</h1>* we are done reading and this is set to close, stop send
* abort upload
* shutting down connection #0
```

4. 404 – Not Found

   Case 1: File not found in ./www

   Purpose: Handling the missing file path.

   Test command:

   curl -v http://localhost:8080/missing.html

   Expected response:

   HTTP/1.1 404 Not Found

   Content-Type: text/html

   Content-Length: ...

   <html><body><h1>404 Not Found</h1></body></html>

   Explanation: The requested file does not exist in ./www.

```
D:\thanh\Fall2025\CMPT371\MiniServer>curl -v http://localhost:8080/missing.html
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
*   Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080
* using HTTP/1.x
> GET /missing.html HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.14.1
> Accept: */*
>
< HTTP/1.1 404 Not Found
< Date: Sat, 25 Oct 2025 21:08:15 GMT
< Server: MiniCartServer/1.0
< Content-Type: text/html
< Content-Length: 22
< Connection: close
<
<h1>404 Not Found</h1>* we are done reading and this is set to close, stop send
* abort upload
* shutting down connection #0
```

   Case 2: Invalid route does not handle by server

   Purpose: Confirm that unrecognized dynamic URLs (not mapped to any logic) also produce 404.

   Test command:

   curl -v http://localhost:8080/randompage

   Expected response:

   HTTP/1.1 404 Not Found

   Date: ...

   Server: MiniCartServer/1.0

   Content-Type: text/html

   Connection: close

   <html><body><h1>404 Not Found</h1></body></html>

Explanation: The resource /randompage is not a file in ./www, and it's not one of the handled routes (add, cart.html)

```
D:\thanh\Fall2025\CMPT371\MiniServer>curl -v http://localhost:8080/randompage
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
*   Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080
* using HTTP/1.x
> GET /randompage HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.14.1
> Accept: */*
>
< HTTP/1.1 404 Not Found
< Date: Sat, 25 Oct 2025 21:08:55 GMT
< Server: MiniCartServer/1.0
< Content-Type: text/html
< Content-Length: 22
< Connection: close
<
<h1>404 Not Found</h1>* we are done reading and this is set to close, stop send
* abort upload
* shutting down connection #0
```

5. 505 – HTTP Version Not Supported

   Purpose: Ensure that only HTTP/1.0 and HTTP/1.1 are accepted

   Test command:

   ```
   $client = New-Object System.Net.Sockets.TcpClient("localhost",8080)
   $stream = $client.GetStream()
   $writer = New-Object System.IO.StreamWriter($stream)
   $request = "GET /test.html HTTP/2.0`r`nHost:
localhost:8080`r`nConnection: close`r`n`r`n"
   $writer.Write($request)
   $writer.Flush()
   $reader = New-Object System.IO.StreamReader($stream)
   $response = $reader.ReadToEnd()
   $response
   $reader.Close(); $writer.Close(); $stream.Close(); $client.Close()
   ```

   Expected response:

   HTTP/1.1 505 HTTP Version Not Supported

   Content-Type: text/html

   Content-Length: ...

   <h1>505 HTTP Version Not Supported</h1>

   Explanation: The request line uses HTTP/2.0 (which is not in SUPPORTED_VERSIONS)

```
PS D:\thanh\Fall2025\CMPT371\MiniServer> $request = "GET /test.html HTTP/2.0`r`nHost: localhost:8080`r`nConnection: close`r`n`r`n"
PS D:\thanh\Fall2025\CMPT371\MiniServer> $writer.Write($request)
PS D:\thanh\Fall2025\CMPT371\MiniServer> $writer.Flush()
PS D:\thanh\Fall2025\CMPT371\MiniServer> $reader = New-Object System.IO.StreamReader($stream)
PS D:\thanh\Fall2025\CMPT371\MiniServer> $response = $reader.ReadToEnd()
PS D:\thanh\Fall2025\CMPT371\MiniServer> $response
HTTP/1.1 505 HTTP Version Not Supported
Date: Sat, 25 Oct 2025 21:15:14 GMT
Server: MiniCartServer/1.0
Content-Type: text/html
Content-Length: 39
Connection: close
```

# III/ Step 3 – Performance:

a.

- Difference of proxy server and a web server:
  Web server hosts and serves local files (e.g. index.html, test.html), take content from the local ". /www" folder. In short, it is the original server that processes HTTP requests directly.
  Proxy server acts as an intermediary that forwards client requests to external servers, take content from external webservers. It is a gateway or caching server that forwards client requests and relays responses.
- Specification:
  1. Proxy server listens for connection on port 8888
  2. It accepts incoming client requests
  3. Then, it extracts the host header to determine the target server
  4. It opens new TCP connection to the destination host (default port is 80)
  5. Forward the original request to that host
  6. Relay the remote server's response back to the client
  7. Close both sockets after the response is complete

  If an error occurs, the proxy will reply with "HTTP/1.1 502 Bad Gateway"

b. Test procedure for proxy server:

- Test Environment:

Proxy port: 8888

Web server port: 8080

1. Case 1 – Successful proxy forwarding:

Purpose: Verify that the proxy receives a client request, connects to the remote host, and relays the full response.

Command:

curl -v -x localhost:8888 http://example.com/

Expected response:

HTTP/1.1 200 OK

Server: ECS (nyb/1D2B)

Content-Type: text/html

Explanation: The HTML received from example.com proves that the proxy is correctly forwarded and returned to the remote response.

```
D:\thanh\Fall2025\CMPT371\MiniServer>curl -v -x localhost:8888 http://exampl
e.com/
* Host localhost:8888 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8888...
*   Trying 127.0.0.1:8888...
* Connected to localhost (127.0.0.1) port 8888
* using HTTP/1.x
> GET http://example.com/ HTTP/1.1
> Host: example.com
> User-Agent: curl/8.14.1
> Accept: */*
> Proxy-Connection: Keep-Alive
>
* Request completely sent off
< HTTP/1.1 200 OK
< Content-Type: text/html
< ETag: "bc2473a18e003bdb249eba5ce893033f:1760028122.592274"
< Last-Modified: Thu, 09 Oct 2025 16:42:02 GMT
< Cache-Control: max-age=86000
< Date: Sat, 25 Oct 2025 21:53:44 GMT
< Content-Length: 513
< Connection: close
< X-N: S
<
<!doctype html><html lang="en"><head><title>Example Domain</title><meta name
="viewport" content="width=device-width, initial-scale=1"><style>body{backgr
ound:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{fo
nt-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style><body><div
><h1>Example Domain</h1><p>This domain is for use in documentation examples
without needing permission. Avoid use in operations.<p><a href="https://iana
.org/domains/example">Learn more</a></div></body></html>
* shutting down connection #0
```

2. Case 2 – Invalid host

Purpose: Ensure that the proxy gracefully reports unreachable or invalid hosts.

Command:

curl -v -x localhost:8888 http://invalid.com/

Expected response:

HTTP/1.1 502 Bad Gateway

Content-Type: text/html

<h1>502 Bad Gateway</h1>

Explanation: Proxy fails to establish a connection to the target.

```
D:\thanh\Fall2025\CMPT371\MiniServer>curl -v -x localhost:8888 http://invalid.com
* Host localhost:8888 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8888...
*   Trying 127.0.0.1:8888...
* Connected to localhost (127.0.0.1) port 8888
* using HTTP/1.x
> GET http://invalid.com/ HTTP/1.1
> Host: invalid.com
> User-Agent: curl/8.14.1
> Accept: */*
> Proxy-Connection: Keep-Alive
>
* Request completely sent off
< HTTP/1.1 502 Bad Gateway
< Date: Sat, 25 Oct 2025 22:17:21 GMT
< Server: MiniCartServer/1.0
< Content-Type: text/html
< Content-Length: 63
< Connection: close
<
<h1>502 Bad Gateway</h1><p>[Errno 11002] getaddrinfo failed</p>* shutting down connection #0
```

3. Case 3 – Local web request bypass

Goal: Verify that the local web server still functions independently.

Command:

curl -v http://localhost:8080/test.html

Expected response:

HTTP/1.1 200 OK

Content-Type: text/html

File test.html from folder ‘./www’

Explanation: Both web and proxy servers can run concurrently on separate threads.

```
D:\thanh\Fall2025\CMPT371\MiniServer>curl -v http://localhost:8080/test.html
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
*   Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080
* using HTTP/1.x
> GET /test.html HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.14.1
> Accept: */*
>
< HTTP/1.1 200 OK
< Date: Sat, 25 Oct 2025 22:19:33 GMT
< Server: MiniCartServer/1.0
< Content-Type: text/html
< Transfer-Encoding: chunked
< Connection: close
```

4. Proxy with If-Modified-Since

Goal: Confirm that proxy transparently forwards conditional GET headers.

Command:

curl -v -x localhost:8888 -H "If-Modified-Since: Fri, 24 Oct 2025 12:00:00 GMT" http://example.com/

Expected response: Proxy forwards the If-Modified-Since header unchanged. If the resource on the remote server has not changed, the remote server responds with 304 Not Modified, which the proxy relays back to the client.

```
D:\thanh\Fall2025\CMPT371\MiniServer>curl -v -x localhost:8888 -H "If-Modified-Since: Fri, 24 Oct 2025 12:00:00 GMT" http://example.com/
* Host localhost:8888 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8888...
*   Trying 127.0.0.1:8888...
* Connected to localhost (127.0.0.1) port 8888
* using HTTP/1.x
> GET http://example.com/ HTTP/1.1
> Host: example.com
> User-Agent: curl/8.14.1
> Accept: */*
> Proxy-Connection: Keep-Alive
> If-Modified-Since: Fri, 24 Oct 2025 12:00:00 GMT
>
* Request completely sent off
< HTTP/1.1 304 Not Modified
< Content-Type: text/html
< Last-Modified: Thu, 09 Oct 2025 16:42:02 GMT
< ETag: "bc2473a18e003bdb249eba5ce893033f:1760028122.592274"
< Cache-Control: max-age=86000
< Date: Sat, 25 Oct 2025 22:23:01 GMT
< Connection: close
```

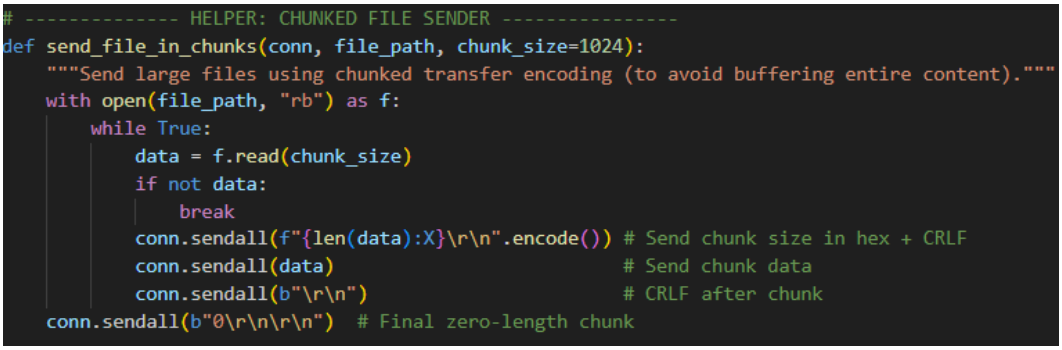
c. Our web server is multi-threaded, which is indicated by the thread part in the while loop:

```
while True:
    conn, addr = s.accept()
    Thread(target=handle_client, args=(conn, addr), daemon=True).start()
```

Each time a client connects, s.accept() returns a new socket connection (conn) and client address (addr). The “Thread(target=handle_client, args=(conn, addr), daemon=True).start()” creates a new thread for the connecting client. daemon=True makes it so that the thread automatically runs when the main program exits. Lastly, the handle_client runs concurrently for each of the connecting clients, so multiple clients can be replied to at the same time. There is also the variable “cart_lock” that enables safe threading by ensuring only one thread can access the shared cart at any time.

If the server were not multithreaded, it would block the handle_client call for each of the connections, making its response time much slower. Therefore, since the server is multithreaded, it is much faster in response to the connections.

## IV/ Step 4 – Expand:



```python
# -------------- HELPER: CHUNKED FILE SENDER ----------------
def send_file_in_chunks(conn, file_path, chunk_size=1024):
    """Send large files using chunked transfer encoding (to avoid buffering entire content)."""
    with open(file_path, "rb") as f:
        while True:
            data = f.read(chunk_size)
            if not data:
                break
            conn.sendall(f"{len(data):X}\r\n".encode()) # Send chunk size in hex + CRLF
            conn.sendall(data)                          # Send chunk data
            conn.sendall(b"\r\n")                       # CRLF after chunk
    conn.sendall(b"0\r\n\r\n")  # Final zero-length chunk
```

HOL Blocking happens when the server operates on an FCFS (First Come First Serve) basis. This means that the requests received first are completely processed before any of the subsequent requests in the same connection are processed. Because of this, if a large file is being transmitted at any given moment, all other requests, regardless of their file sizes, are delayed until the first file is completely transmitted. In our code, we send larger files in the format of small chunks.
The “send_file_in_chunks” helper splits the large file into sections of 1024 bytes. Instead of waiting to build the whole file in memory and send it all at once, the server streams data in chunks/frames over the connection. Because of this, smaller pieces of data are sent immediately, and if multiple requests arrive at the same time, the server can start sending some chunks of the first request while starting to read and queue the other requests (parallelization). Without this, the performance in the case of multiple requests would be much slower, as the clients would have to wait for the entire large file to be transmitted first.